bool datatype:

We can use this datatype to represent boolean values
The allowed values for boolean data type are : True, False

is eligible -> Yes (True)
is not eligible --> No (False)

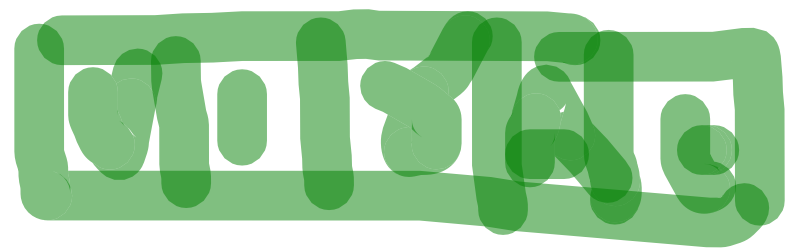
Is customer active --> yes or no --> True or False
a = True
status = True
is_eligible = True
is_active = True

internallly **True** means 1 and **False** means 0

string datatype :

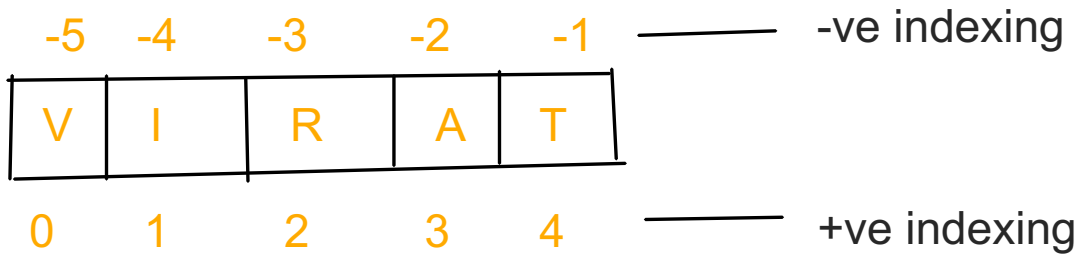
str is the representation of string datatype
A string is sequence of characters, enclosed within a single quotes or double quotes also
it can have a alphanumberic values as welll

favourite_cricketer = "virat"
customer_name = "dhoni MS"
order_info = "ordered selected"
email_id = "test@gmail.com"



"Python class is very and useful" for java students also'"

cricketer= "virat"



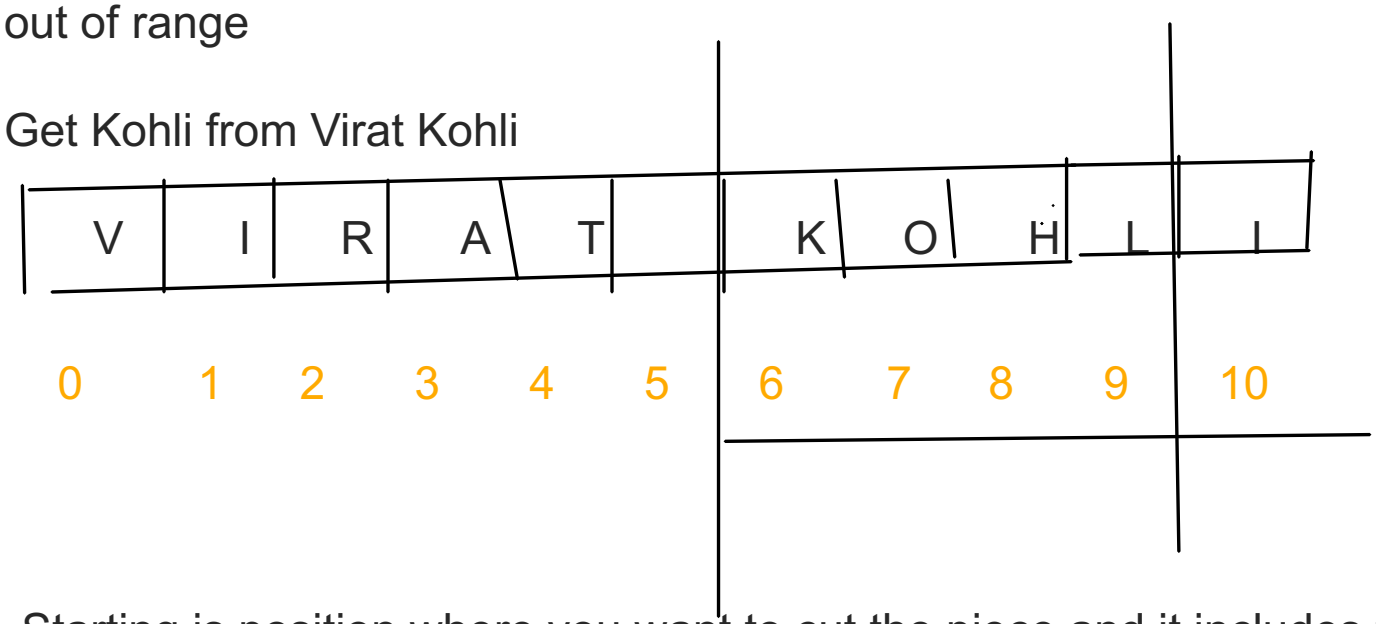
Slicing of Strings:
slice means a piece of big portion
[ ] is it s slicing operator, which can used to retrieve parts of string

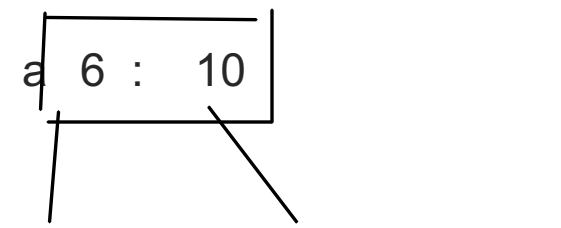
In python string follows indexing mechanism to store characters
Index always starts from zero (0,1,2,3,4,.....)

In python indexes can be postitve or negative
+ve index means forward direction from left to right
-ve index means backward direction from right to left

Note: if we try to fetch value using the index more than the indexed position then we get IndexError: string out of range

Get Kohli from Virat Kohli





Starting is position where you want to cut the piece and it includes that position character
endpoisition is where you want to end the piece but ending index **chracter wont by inclusive**
If we dont give any endposition, ending index will be treated as the end of entire string

Note:
In python int, float, complex, bool and str are the fundamental datatypes

TYPE CASTING:
a = "10"
b = "20"

The process of converting one type to another type is nothing but a typecasting or type coersion.
These are the following functions for typecasting
1. int() :
We can use this function to convert values from other type to int

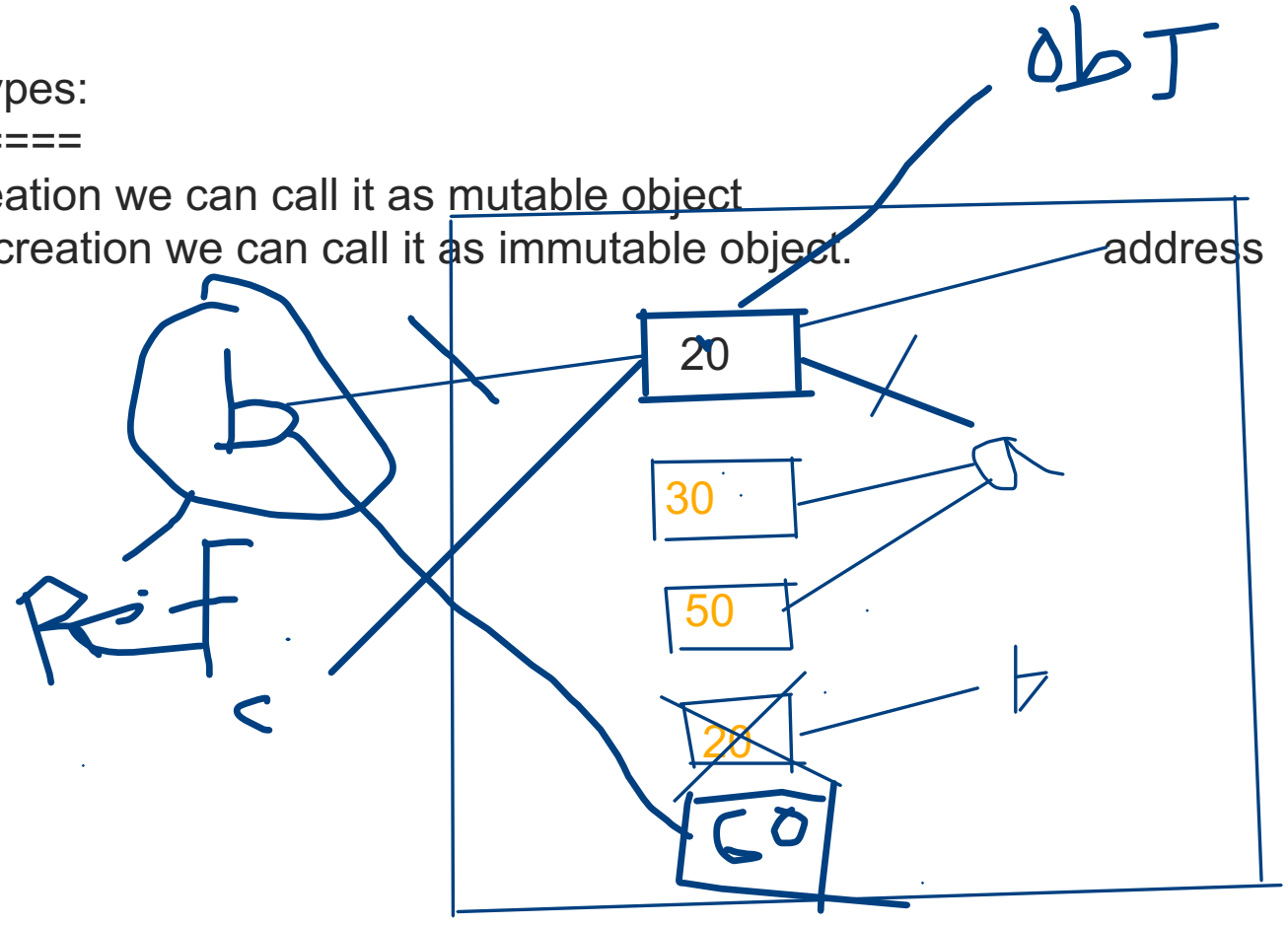
Note:
1. We can convert any type to int except complex type
2. If we want to convert string to int type, mandatorily string should have only integral values and that should have base-10, otherwise it will get ValueError

2. float() --> We can use float function to convert other types(int, bool, str, etcc) to float type
Note:
1. We can convert any type to float except complex type
2. If we want to convert string to float type, mandatorily string should have only floating or integral values and that should have base-10, otherwise it will get ValueError

3. complex() --> We can use this function to convert any other type to complext type

4. bool() --> We can use this function convert any toher type to bool
5. str() --> We use this function to convert any other type to string

Immutability w.r.to Fundamental DataTypes:
==========
If I'm able to change the object after creation we can call it as mutable object
If I'm unable to change the object after creation we can call it as immutable object.



In python if a new object is required, PVM wont create new object immmediately rather it will check the exist object, if any object has same content or data, it will reuse that object.

If existing objects are not having same content, then only it will create a new object

Advanced Datatype or datastructures:

bytes
bytearray
list
tuple
set
range
dict

bytes datattype :

bytes data type is helpful to represent a group of numbers, like a an array
the only allowed values are 0 to 255, accidentally if we give more than this range then we will get valueerror

Syntax:
we have to use bytes(x) --> x is having sequence of numbers
Ex: bytes([10,20,30,40])

bytes is immutable we can not change its values or reassign once it is created, if we try to modify we will get type error
if we want to store data within 0 to 255 and i dont want anyone to be able to modify/change after creation, then bytes is right choice

1, 2 3 4 5 6 7 8 9, 10, ... 23 --> state codes in vodafone

bytearray data type:
bytearray is exactly same as bytes datatype except that it is mutable(i.e we can change the values)

the only different between byte and bytearray is bytes is immutable and bytearray is mutable

list datatype:
If we want to store group of values in a single variable, where values having different type of data or same type of data

marks = [10,20,30,40] --> Same type of data, this is possible (homegenious data)
student_data = [12345, "kasi yedumati", 98, True] --> Different type of data(heterogenious data)

1. It maintains insertion order
2. it allows heterogenious(different type of values) data
3. duplicate data also allowed
4. it is growable in nature(we can add data after creation or remove data or modify data or fetch data)
5. we should create list variable values using [ ]
Ex: sutdent_list = ["kasi", "virat","dhoni"]

student_names = ["kasi", "bhuvan", "chandu", "varshith", "harika", "chandu"]

student_name[3] --> varshith

Note: List is an ordered, mutable, heterogenious collection of elements and also duplicated are ordered

tuple data type:

tuple is exactly same as list, except that it is immutable i.e we cannot change the values once it is created

syntax : we have to create a tuple using paranthesis ( )

Ex: (10,20,30)

Note: tuple is the read only version of list

range datatype:

range datatype represents sequence of numbers

rang is also immutable, the values/numbers present range datatype are not modifiable.

Syntax1:

range(x) --> x will be a number

range(20) --> it generates numbers from 0 to 19

Syntax 2:

range(start, end) --> start will be a number and end also should be a number

Syntax3:

range(start, end, increment)

start --> it is where i want to sequence range should start from

end --> it is where i want to stop range

increment --> how many values you have to increment for every generate sequence numbers

range(10, 20, 2)

Set datatype:
Whenever we want to store group of values where insertion order is not required and duplicated are not allowed, in that case we go for set

1. insertion order is not preserved
2. duplicates are not allowed
3. hetergenious objects are allowed (same or different type of values)
4. it doesnt follow indexing
5. it follows hashing mechanism to insert or update data or manage data in set
6. it is mutable (once the object is created we can modify)
7. it is growable in nature

fruits_set = {'banana', 'apple'}
fruits_set = set() --> create an empty set

frozenset Datatype:
frozeznset is exactly same as set , except that it is immutable.
Once it is created, we cant add, remove or modify its items.

Syntax:
frozenset([10,20,30])

dict datatype :
If we want to store a group of values as **key-value** pairs then we should go for dictionary datatype

Syntax:
contacts = {"name" : "mobilenumber"}

employee_data = ["kasi", 1234, 123000, "hyd"]   --> list data form
employee_data =
{
  "name" : "kasi",
  "employeeid" : 12345,
  "salary" : 12345.50,
  "location" : "hyd"
}

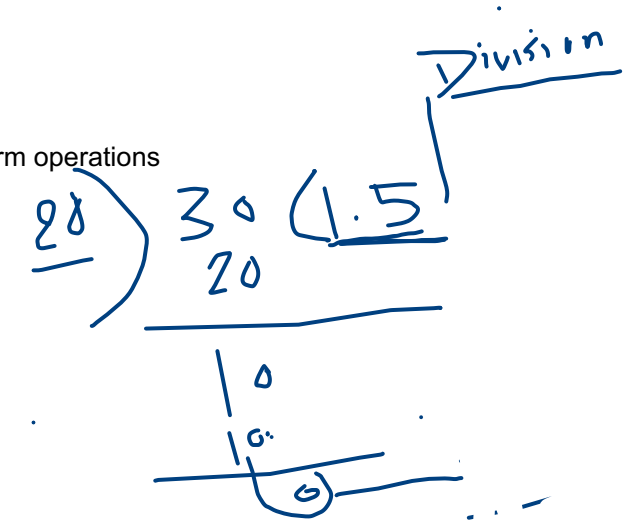
Key must be unique
value can be unique or duplicate
immutable datatype (string, tuple, frozenset)
Using key we can access the data

Values can be any datattype

if we try to add same key twice, it will just consider latest entry value or it overrides values within the same key
it wont throw any error

Datatype	Duplicates	Insertion order	heterogenious	mutable or immutable	syntax	follows index?
int	not applicable	not applicable	only integers 0 to 9	immutable	a = 10; b = 20	
float	not applicates	not applicable	only floating values	immutable	a= 10.5	
complex	not applicable	not applicable	real and imaginary	immutable	a = 3+0j	
bool	not applicable	not applicable	True, False	immutable	a = True ; a= False	
string	allowed	applicable	No	immutable	a= "hello"	Yes, eg: a[2]
bytes	allowed	applicable	No(0 to 255)	immutable	b = bytes([10,20,30]}	Yes
bytearray	allowed	applicable	No	mutable	b =bytearray([10,20]}	Yes
list	allowed	applicable	Yes	mutable	names=['shreyas','sai']	Yes, eg: names[1]
tuple	allowed	applicable	Yes	Immutable	x=(1,3+2j, 'chandu')	Yes, eg: x[2][2]
range	allowed	applicable	No	mutable	x=range(0,9); 0,1,2,3 4,5,6,7,8	Yes
set	not allowed	not preserved	Yes	mutable	x={1,2,3,4}	No (because it follows hashing)
frozenset	not allowed	not preserved	Yes	Immutable	x=frozenset([1,2,3]}	no
dict	duplicate keys are not allowed	applicables	Yes	Mutable	d={'a' : 1, 'b':2}	we access values with key

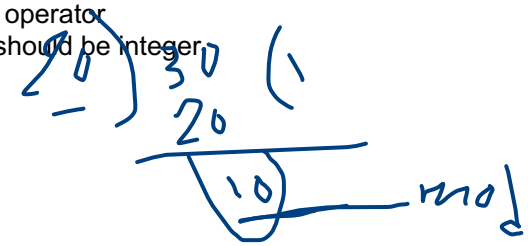
Operators:
Operator is a symbol or keyword using which we can perform operations



1. Arithmatic operators
+ --> Addtion/sum values
- -> subtraction
* -> multiplication
/ -> Division operator
% --> modulo operator (it returns the reminder

// --> Floor division operator
** -> Power operator or exponential operator

+ operator we can against strings also, we call it as concatination operator
* operator also we can use against strings, but alteast one value should be integer.
String multiplication operator



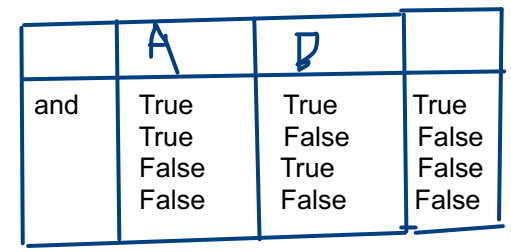
2. Relational or Comparision operators
>
<
>=
<=

Comparision operators will compare any two values and return boolean values True or False
Equality operators --> == !=

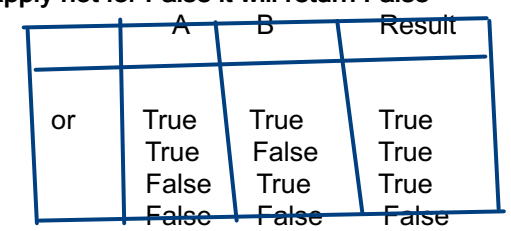
3. Logical operators
There are three logical operators: **and , or , not**
These keywords/operators we can apply on all datatypes

These operators also check and return **boolean value**

and --> If both arguments are **True** then only result is **True**
or --> If atleast one argument is **True** then output will be True
not --> complement(if we apply not for True it will become False, if we apply not for False it will return False



	A	result
not	True	False



0 means False
non-zero means True
empty string is always treated as False

if a is evaluates to False return a otherwise it returns y

4. Assignment operators
5. Bitwise operators
6. Special operators